

Linked lists (LLs)

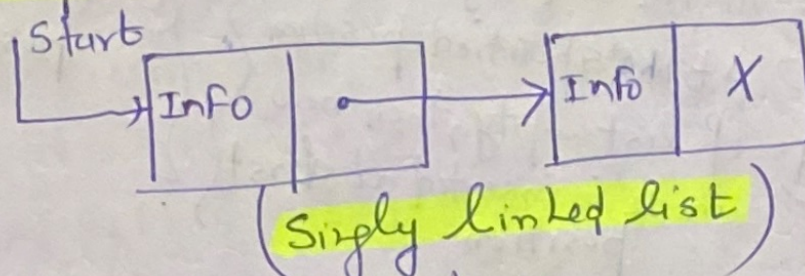
(1)

if the memory allocated ^{is done} at the compile time in a program, it is fixed and cannot be changed during execution of program.

Hence, we need some adaptive method of Mm allocation that gives more flexibility during execution of program. This flexibility can be obtained with special data structure called

linked lists (LLs).

→ In LLs, the logical ordering is represented by having each element pointing to next element. Each element is called a node which has two parts: INFO part, which stores information and a pointer part which points to next element.



Advantages

LLs have many advantages:

- (1) Linked lists are dynamic data structures: That is, they can grow or shrink during the execution of a program.
- (2) Efficient Mm utilization: Mm can be allocated, whenever it is required and deallocated, whenever it is no longer needed.
- (3) Insertion and deletions are easier and efficient: LLs provide flexibility in inserting a data item at a specified position and deletion of a data from given position.
- (4) Many complex applications can be easily carried out with LLs.

Representation of Linear LL

```
struct node  
{  
    int a;  
    struct node *next;  
};  
typedef struct node NODE;  
NODE *start;
```

It defines a new data type whose each element is of type node-type and gives it a name node.

Operations on LLs

The basic operations to be performed on LLs are:

- (1) Creation (2) Insertion (3) Deletion (4) Traversing
- (5) Searching (6) Concatenation (7) Display

This operation is used to create a LL, whenever it is required/needed.

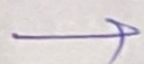
This operation is used to insert a new node in LL at specified position.

A ^{new} node can be inserted:

- (a) At the beginning of list
- (b) At the end of LL
- (c) At the specified position
- (d) if list empty, then new node is inserted at first position

This operation is used to delete an item (node) from LL. A node may be deleted from:

- (a) Beginning of LL
- (b) End of LL
- (c) Specified position



Traversing is a process of going through all the nodes of LL from one end to other end.

→ Concatenation is a process of appending (joining) the second list to the end of first list. The last node of first list is modified so that it can now point to first node of second list.

→ Display is used to print each and every element / node's information.

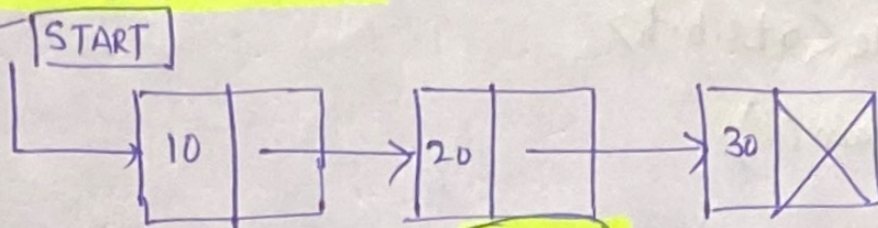
Types of LLs

- (a) Singly Linked list (SLL)
- (b) Doubly linked list (DLL)
- (c) Circular LL (CLL)
- (d) Circular Doubly LL (CDLL)

Singly linked list [Pictorial representation]

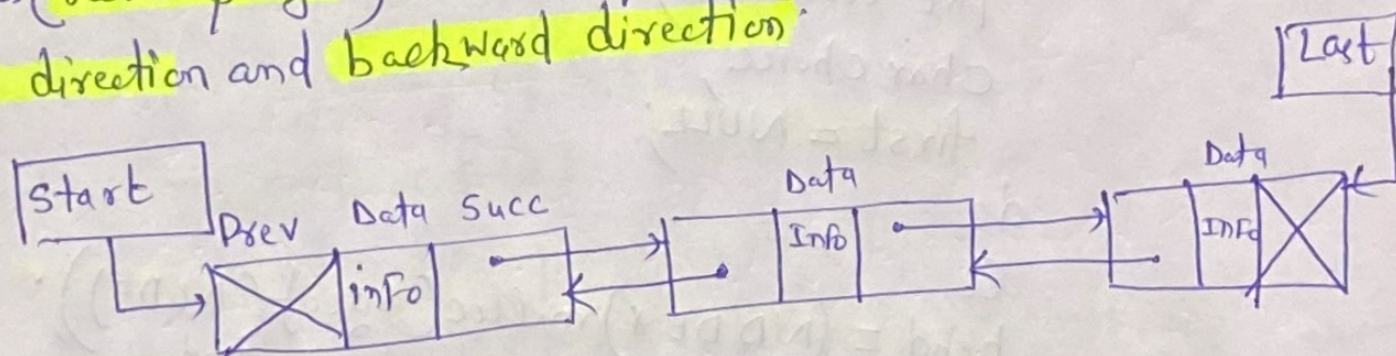
(2)

- ① A SLL is one in which all nodes are linked together in some sequential manner. Hence it is called Linear linked list.

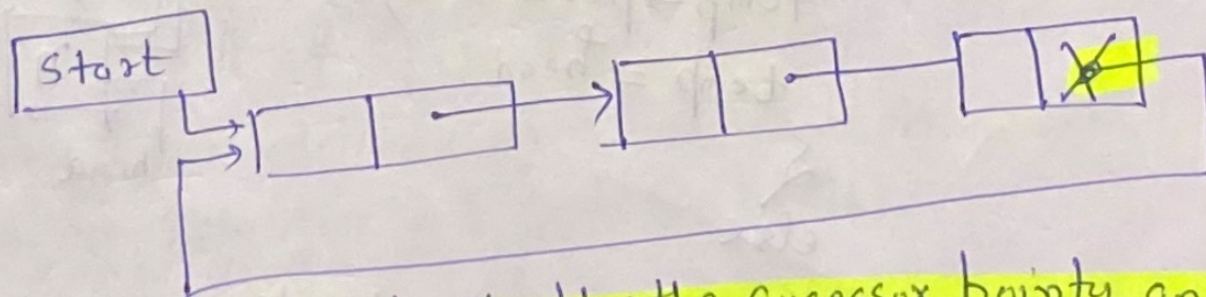


The main problem with SLL is that we cannot access the predecessor node from current node.

- ② A DLL is one in which all the nodes are linked together by multiple links which help in accessing both successor node (next node) and predecessor node (previous node) for any arbitrary node within the list. Therefore, each node in DLL field contains two pointers (left & right). This helps to traverse the list in forward direction and backward direction.



- ③ A Circular linked list (CLL) is one which has no beginning and end. A SLL can be made a CLL by simply storing the address of first node in the link field of last node.



- ④ A CDLL is one which has both the successor pointer and predecessor pointer in circular manner.

